

ML Problem Solving

K-Nearest Neighbor (KNN) Classifier & Principal Component Analysis (PCA)

Part 1: Feature Space Visualization & Dataset Introduction

Consider the following training dataset D , where each row represents an observation. The first two columns are input features (x_1, x_2) , and the true class label is the final column.

$$D = \{A : [3, 3, 1], B : [1, 1, 1], C : [-1, 0, 1], D : [2, 2, 0], E : [-2, 2, 0]\}$$

Questions:

- Coordinate Mapping:** Draw the data points in a 2D Cartesian coordinate system to represent your input space. Mark points belonging to Class 1 with an 'X' and Class 0 with an 'O'. Label each point clearly (e.g., A, B, C, D, E).
- Geometric Intuition:** Inspect the plot visually. Do you notice any natural clusters or potential linear separability issues? How might a non-parametric method like KNN handle this layout compared to a linear classifier?

Part 2: Distance Metrics & Distance Matrices

Distance computation is the core of lazy learners like KNN. Different distance metrics can drastically alter neighborhood composition and final predictions.

The **Euclidean Distance** between two points $p = (p_1, p_2)$ and $q = (q_1, q_2)$ is defined as:

$$d_{\text{euclidean}}(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}$$

The **Manhattan Distance** (Taxicab metric) is defined as:

$$d_{\text{manhattan}}(p, q) = |p_1 - q_1| + |p_2 - q_2|$$

Questions:

- Euclidean Matrix:** Create a 5×5 distance matrix for the dataset D using Euclidean distance. (Note: This is a symmetric matrix where the diagonal elements are 0).
- Manhattan Matrix:** Create a separate 5×5 distance matrix for dataset D using Manhattan distance.
- Metric Comparison:** Compare the two matrices. Did changing the distance metric alter the relative ranking of nearest neighbors for any of the points? Explain why distance metric choice matters in high-dimensional feature spaces.

Part 3: Training Evaluation, Confusion Matrices, and the Role of K

When evaluating a model on its training set, every point being evaluated is already present inside the dataset. Let's explore how KNN performs across different values of k when evaluated directly on D .

Questions:

6. **In-Sample Predictions:** Using the Euclidean distance matrix, predict the classes using the provided training datapoints themselves as queries for $k \in \{1, 3, 5\}$. (Note: When evaluating a point against itself, its distance is 0, so the nearest neighbor of a point is itself if included in the pool).
7. **Training Accuracy:** Write down the training accuracy for each k . Which k yields the highest training accuracy? Why does this occur?
8. **Confusion Matrix Construction:** For each $k \in \{1, 3, 5\}$, draw or format a 2×2 confusion matrix (with rows representing predicted classes and columns representing actual classes).
9. **Extreme Values of K :**
 - What happens theoretically to the decision boundary and training error when $k = 1$?
 - What happens when $k = n$ (where n is the total number of training points, i.e., $n = 5$)? Discuss underfitting and overfitting in these contexts.

Part 4: Generalization: Training vs. Testing Data & Out-of-Sample Prediction

So far, we evaluated points that *were* present in the training set. Now, let's examine what happens when we introduce genuinely new, unseen test points where the point is **not** present in the training data.

Consider the new test set T :

$$T = \{F : [3, 1, 1], G : [0, 2, 0]\}$$

Questions:

10. **Out-of-Sample Prediction:** Calculate the distances from each test point in T to all points in training set D . Predict their classes for $k \in \{1, 3, 5\}$ using Euclidean distance.
11. **Did the ‘Best’ K Change?** Compare the behavior of different k values on training data versus test data T . Did the optimal choice of k change? If so, why do we use validation sets or cross-validation instead of optimizing k on training accuracy?
12. **The Presence Paradox:** Explain the conceptual difference between making predictions when the tested point is present in the training data versus when it is an unseen test point. How does the inclusion of the test point itself (aliasing/self-matching) affect distance ranking and tie-breaking when $k = 1$?

(optional) Part 5: Dimensionality Reduction via Principal Component Analysis (PCA)

In many machine learning applications, feature spaces grow to hundreds or thousands of dimensions, making distance-based algorithms like KNN vulnerable to the *curse of dimensionality*. Principal Component Analysis (PCA) is an unsupervised linear transformation technique used to maximize variance and reduce dimensionality.

Consider only the input features of dataset D (ignoring the class labels for a moment):

$$X = \begin{bmatrix} 3 & 3 \\ 1 & 1 \\ -1 & 0 \\ 2 & 2 \\ -2 & 2 \end{bmatrix}$$

Questions:

13. **Data Centering:** Compute the mean vector $\mu = [\mu_1, \mu_2]$ of the feature matrix X . Subtract the mean from each observation to obtain the centered data matrix X_{centered} . Why is mean-centering a mandatory preprocessing step before running PCA?
14. **Covariance Matrix:** Compute the 2×2 sample covariance matrix C of the centered data. What do the diagonal and off-diagonal elements of C represent?
15. **Eigenvalues and Eigenvectors:** Find the eigenvalues (λ_1, λ_2) and corresponding orthonormal eigenvectors (v_1, v_2) of the covariance matrix C . (You may show the characteristic equation or use standard linear algebra steps). Which eigenvector represents the first principal component (PC1)?

16. **Explained Variance Ratio:** Calculate the fraction of total variance explained by the first principal component (PC1). How much information (variance) is retained if we reduce our 2D feature space down to a 1D subspace along PC1?
17. **Projection & Transformation:** Project the centered data matrix X_{centered} onto the first principal component eigenvector v_1 to obtain a new 1D representation Z .
18. **PCA Impact on KNN:** Discuss how projecting data onto principal components prior to applying KNN might affect distance calculations, noise reduction, and overall classification performance. What are the potential risks of throwing away lower-variance principal components?